

YENEPOYA INSTITUTE OF ARTS, SCIENCE, COMMERCE AND MANAGEMENT

Project – Final Project Report

on

End-to-End Encryption for Manufacturing Cloud Apps

Student Name: Pranith Shetty B

Register Number: 23BCCE128

**Course: BCA in Cloud Computing, Cyber Security &
Ethical Hacking with IBM**

Industry Mentor: Mr. Shashank

Guided by: Ms. Vijayalaxmi

Table of Content

Section	Title	Page
	Executive Summary	1
1	BACKGROUND	
1.1	Aim	2
1.2	Technologies Used	2-3
1.3	Hardware Architecture & Network Setup	4
1.4	Software Architecture & Framework	5-6
2	SYSTEM	
2.1	Requirements Specification	7-12
2.2	System Design & Architecture	12-17
2.3	Implementation (Flask + Fernet + Database)	18-19
2.4	Testing & Validation	20-22
2.5	Graphical User Interface (GUI)	22-25
2.6	Customer Testing	26
2.7	Evaluation	27
3	PROJECT SNAPSHOTS (UI & OUTPUT SCREENS)	28-34
4	CONCLUSION	35
5	FUTURE DEVELOPMENT	36-37
6	REFERENCES	37
7	APPENDIX	
7.1	Sample Encrypted Messages	38
7.2	Database Schema & Tables	39

7.3	System Output Screens	39
7.4	Glossary of Terms	40
7.5	Abbreviations	41

Executive Summary

SecureManufPro is a web-based secure messaging application designed for manufacturing environments, where protecting sensitive production data is critical. In many manufacturing systems, communication between users can be vulnerable to unauthorized access, which can lead to data leakage and operational risks. This project addresses that challenge by implementing end-to-end encryption (E2EE) using the Fernet cryptographic algorithm.

In this system, whenever a user sends a message, it is automatically encrypted before being stored in the database. This ensures that even if the stored data is accessed without permission, it remains unreadable. Only the intended receiver, using the correct secret key, can decrypt and view the original message.

The application supports multiple user roles, including Admin, Operator, QC Inspector, and Maintenance, each with defined access permissions. The system includes key features such as user registration with admin approval, secure login, encrypted messaging, role-based access control, audit logging, and login activity tracking to enhance security monitoring.

The project is developed using Python with Flask for backend processing, SQLite for database management, and HTML, CSS, and JavaScript for the frontend interface. Testing confirms that encrypted messages stored in the database cannot be interpreted without proper decryption, ensuring strong data confidentiality.

Overall, SecureManufPro demonstrates how modern encryption techniques can be effectively applied in manufacturing cloud applications to secure communication and protect sensitive information in a reliable and scalable manner.

1. BACKGROUND

1.1 Aim

The main aim of this project is to develop a **secure messaging system for manufacturing environments**, where communication between workers plays a critical role in daily operations.

In a manufacturing unit, important information such as machine status, quality inspection results, and maintenance alerts is shared frequently. If this data is accessed by unauthorized users, it can lead to serious operational issues or data leakage.

To solve this problem, the system ensures that **every message is encrypted before it is stored or transmitted**. Only the intended sender and receiver can view the original message by using a secret key.

Additionally, the system implements **role-based access control**, where different users such as Admin, Operator, QC Inspector, and Maintenance staff have different permissions based on their responsibilities. This ensures that users can only access information relevant to their role.

1.2 Technologies Used

The development of SecureManufPro involves a combination of backend, frontend, and security technologies that work together to deliver a secure and efficient messaging system. Each technology plays a specific role in ensuring the functionality, security, and usability of the application.

The backend of the system is developed using Python, which provides a strong and flexible programming environment for handling application logic. Flask is used as the web framework to manage routing, handle user requests, and connect the frontend with the backend services.

To ensure data security, the system uses the Fernet encryption algorithm, which provides symmetric encryption for protecting sensitive messages. This ensures that all

messages are securely encrypted before being stored and can only be decrypted by authorized users.

For data storage, SQLite3 is used as a lightweight and efficient database solution. It stores user credentials, encrypted messages, and system logs in a structured format.

The frontend of the application is built using HTML5, CSS3, and JavaScript. HTML5 defines the structure of the web pages, CSS3 enhances the visual appearance, and JavaScript enables dynamic interaction between the user and the system.

Technology Stack Overview

Technology	What it does in simple words
Python	The main language used to write the backend logic
Flask	A tool that helps create website addresses like /login or /send
Fernet	A lock that scrambles messages so only the right key can open them
SQLite3	A small database that stores all users and messages
HTML5	Creates the structure of web pages like boxes and buttons
CSS3	Adds colors, fonts, and makes the page look nice
JavaScript	Makes buttons work and sends data without reloading the page

1.3 Hardware Architecture & Network Setup

The project follows a client-server architecture, where a central server manages all operations and multiple client devices access the system through a network. This architecture ensures better control, improved security, and efficient communication within the manufacturing environment.

The server acts as the main processing unit of the system. It runs the backend application and handles tasks such as user authentication, message encryption, decryption, and database operations. All data is processed and stored securely on the server.

The client devices are used by different users such as Admin, Operator, QC Inspector, and Maintenance staff. These devices access the system using a web browser, making the application easy to use and accessible from different platforms without requiring installation.

The network connects the server and client devices. This can be implemented using a Local Area Network (LAN), Wi-Fi, or Ethernet connection. A stable network ensures smooth data transmission between users and the server.

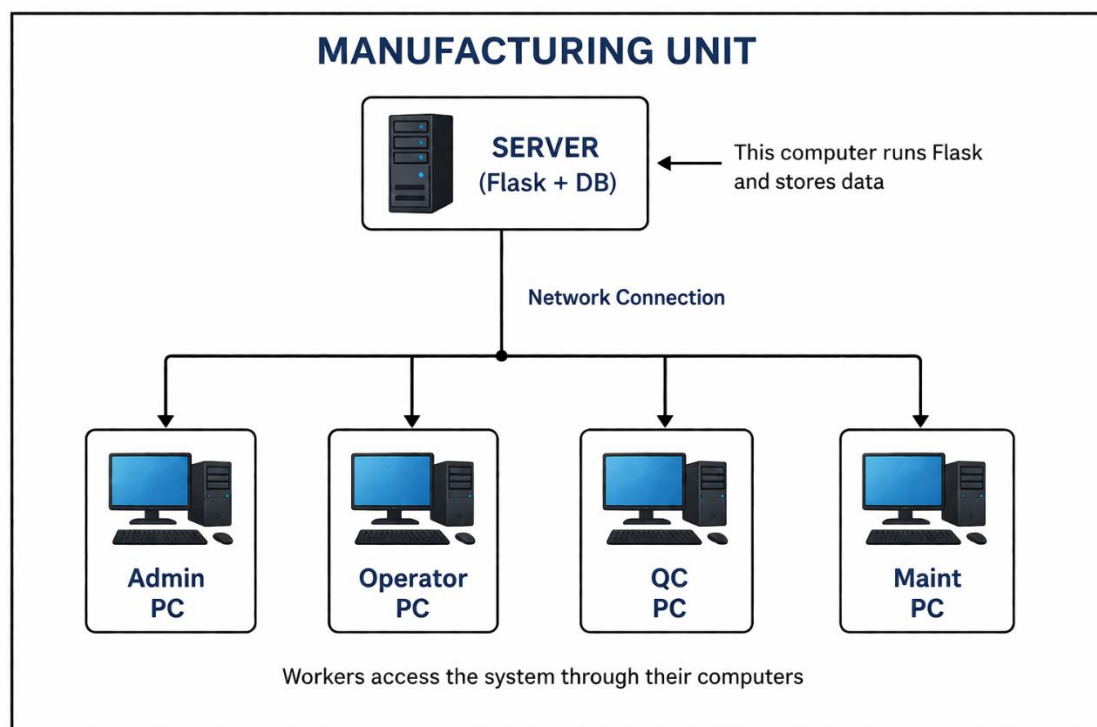


Fig 1: Manufacturing Unit Network Architecture

1.4 Software Architecture & Framework

The project is designed using a **three-layer architecture**, which separates the system into different levels based on functionality. This structure improves system organization, maintainability, and scalability.

Each layer has a specific role and communicates with other layers to perform the complete operation.

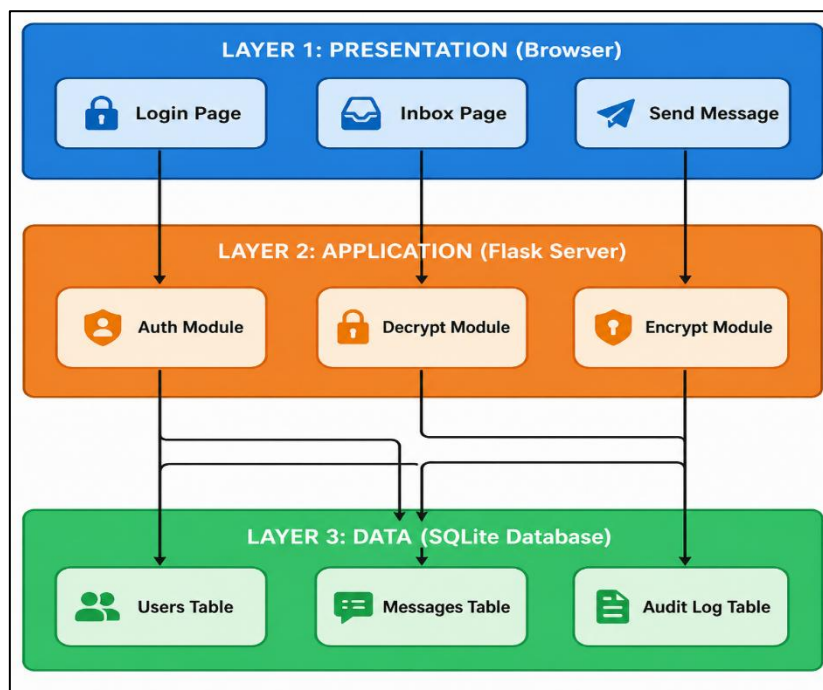


Fig 2: Three-Tier Architecture of Secure Messaging System

Layer Description

1. Presentation Layer (Frontend)

This layer is responsible for user interaction. It includes web pages such as login forms, dashboards, and message interfaces. Users perform actions like sending messages or viewing data through this layer.

2. Application Layer (Backend)

This layer acts as the core of the system. It processes user requests, validates input, performs encryption and decryption, and enforces access control. It ensures that only authorized users can perform specific actions.

3. Data Layer (Database)

This layer stores all system data, including user details, messages, and logs. All sensitive messages are stored in encrypted format to maintain confidentiality.

2. SYSTEM

2.1 Requirements

2.1.1 Functional Requirements

Functional requirements describe the essential features and operations that the system must perform. In this secure messaging system for the manufacturing unit, these requirements define how users interact with the system, how data is processed, and how security is maintained through authentication, encryption, and controlled access.

These requirements ensure that only authorized personnel can access the system, all communications remain encrypted, and all activities are properly monitored.

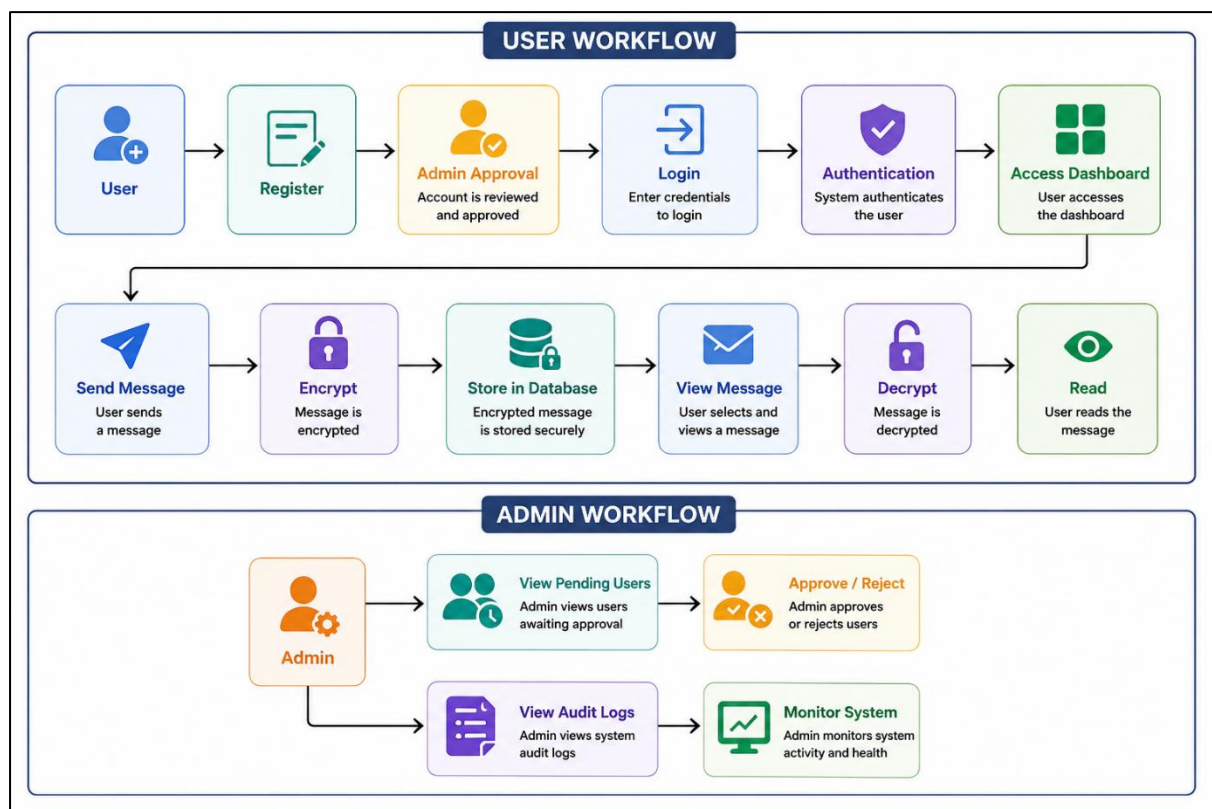


Fig 3: Functional Workflow of Secure Messaging System

The functional workflow diagram illustrates how users interact with the system. A user first registers and waits for admin approval. After successful login and authentication, the user can

send encrypted messages or view received messages by decrypting them. The admin manages user approvals and monitors system activity through audit logs.

Functional Requirements Table

ID	Requirement	What it means for the manufacturing unit
FR-01	New users can register with full name, username, password, and role	A new worker joining the factory can create their own account
FR-02	Passwords are hashed using SHA-256 before storing	Even if someone sees the database, they cannot know the actual password
FR-03	New accounts are saved with status "pending"	A new worker cannot login until the supervisor approves
FR-04	Admin can view all pending user registrations	Supervisor can see a list of all workers waiting for approval
FR-05	Admin can approve or reject pending users	Supervisor can allow or deny access to any worker
FR-06	Users can login with username and password	Workers can access the system using their credentials
FR-07	System checks account status during login	If a worker is not approved, they cannot login
FR-08	System creates a session after successful login	The system remembers the worker is logged in
FR-09	Session ends on logout or after 30 minutes of no activity	If a worker forgets to logout, the system logs them out automatically
FR-10	Every message is encrypted using Fernet before storage	A production alert becomes unreadable to outsiders
FR-11	Plaintext messages are never stored in the database	Database only contains encrypted data
FR-12	Users can send encrypted messages to other users	Operator can send a message to QC about batch quality
FR-13	Users can select priority level for each message	Critical issues marked high priority

FR-14	System shows messages where user is sender or receiver	Workers only see relevant messages
FR-15	Users can click on a message to decrypt and read it	Encrypted message becomes readable on request
FR-16	System checks authorization before decrypting	Unauthorized users cannot access messages
FR-17	Messages are marked as read after decryption	Opened messages are tracked
FR-18	Unread message count shows in the sidebar	Workers can easily track new messages
FR-19	Dashboard shows statistics like total and unread messages	Quick overview of system activity
FR-20	All important actions are logged in audit log	Every action is recorded for security
FR-21	Every login attempt is recorded with timestamp	Admin can track login activity
FR-22	Admin can view complete audit log history	Helps in monitoring and investigation
FR-23	Admin can delete all messages from database	Used for system maintenance
FR-24	Machine status page shows all equipment	Workers can monitor machine availability

2.1.2 User Requirements

Definition:

User requirements describe what different users (Admin, Operator, QC Inspector, and Maintenance staff) expect from the system and the actions they are allowed to perform based on their roles.

User Roles in Manufacturing Unit

Role	Who they are	What they do in the factory
Admin	Plant Supervisor / IT Manager	Manages users and monitors system activity
Operator	Machine Operators	Operates machines and sends production updates
QC Inspector	Quality Control Staff	Checks product quality at QC stations
Maintenance	Repair Engineers	Handles machine breakdown and maintenance

User Requirements Table

ID	User	What they can do
UR-01	All users	Login, logout, and access dashboard with message statistics
UR-02	All users	Send encrypted messages with receiver selection and priority
UR-03	All users	View, filter, and decrypt messages in inbox
UR-04	All users	View machine status information
UR-05	New users	Register account and wait for admin approval
UR-06	Admin	Approve/reject users and manage access
UR-07	Admin	View audit logs and monitor system activity
UR-08	Admin	Delete messages for system maintenance
UR-09	Operator	Access machine-related information (Machine A, B, C)
UR-10	QC Inspector	Access quality control station data
UR-11	Maintenance	Access maintenance and repair information

2.1.3 Environmental Requirements

Definition:

Environmental requirements specify the hardware, software, and network conditions needed for the system to function properly in the manufacturing unit.

Hardware Requirements

Component	Minimum Needed	Recommended
Server Computer	Dual Core processor, 4GB RAM	Quad Core processor, 8GB RAM
Server Storage	10GB free space	20GB free space
Worker Computer	Any computer with web browser	Core i3 or better
Network Speed	10 Mbps connection	100 Mbps or faster

Software Requirements

Component	What is required
Server OS	Windows 10, Linux Ubuntu, or macOS
Python Version	3.10 or higher
Web Browser	Chrome, Firefox, Edge, or Safari
Database	SQLite3 (built-in with Python)
Flask	Version 2.3.0
Cryptography	Version 41.0.0

Network Requirements

Requirement	Explanation
Server IP Address	Fixed IP so workers can always access the server
Network Port	Port 5000 must be open on server firewall
Worker Access	All worker computers must connect to server
Production Security	HTTPS with SSL certificate for secure communication

2.2 Design Blueprint & Architecture

The design of the Secure Messaging System for the manufacturing unit follows a modular and layered architecture. The system is divided into multiple components, where each module performs a specific function such as user authentication, message encryption, data storage, and user interface display.

The process begins when a user accesses the system through a web browser and logs in using their credentials. The Flask server handles authentication by verifying the username and password. If the user is approved, access to the dashboard is granted.

When a user sends a message, the system encrypts the message using the Fernet encryption algorithm before storing it in the database. This ensures that all sensitive information remains secure.

When another user views the message, the system checks authorization and decrypts the message only if the user has permission. All activities such as login, message sending, and decryption are recorded in the audit log.

The SQLite database stores user data, messages, and logs, while the Flask backend processes requests and communicates with the frontend interface.

This modular design ensures security, scalability, and efficient communication within the manufacturing unit

System Components Table

Component	Function
User Interface (Browser)	Allows users to interact with system
Flask Backend	Handles requests and application logic
Authentication Module	Verifies user login and access
Encryption Module	Encrypts messages before storage
Decryption Module	Decrypts messages for authorized users
SQLite Database	Stores users, messages, and logs
Audit Log System	Tracks all system activities

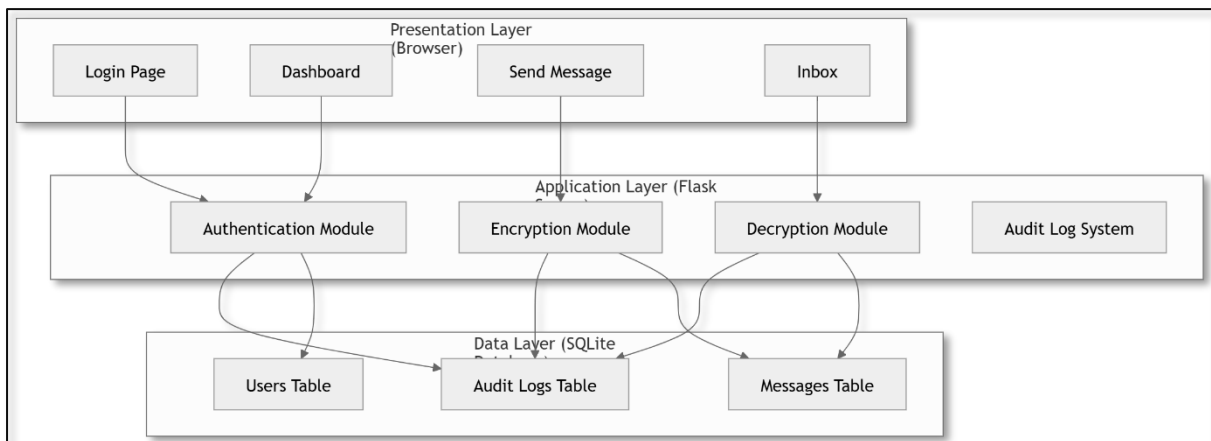


Fig: Design Blueprint & Architecture

2.2.1 Use Case Diagram

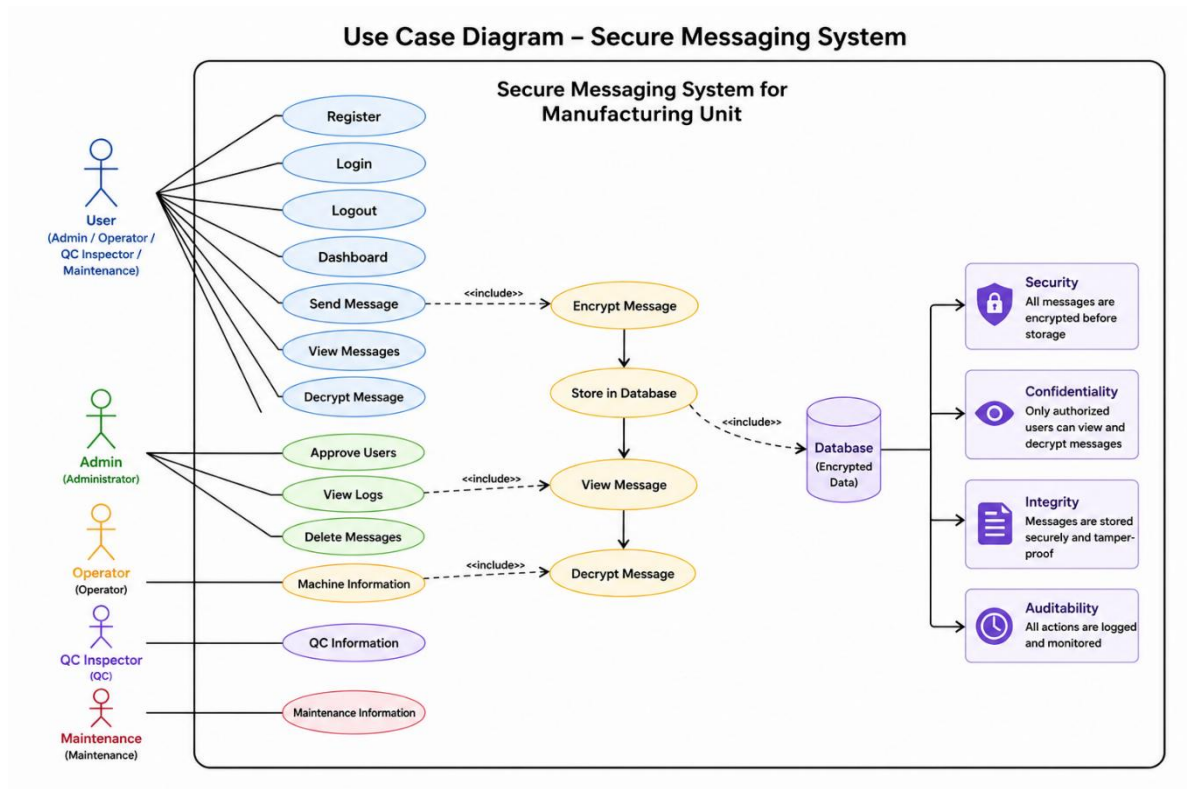


Fig: Use Case Diagram of Secure Messaging System

The use case diagram represents the interaction between different users and the system. All users can perform basic actions such as login, sending messages, and viewing messages. The admin has additional privileges like approving users, viewing audit logs, and deleting messages. Role-specific users such as operators, QC inspectors, and maintenance staff can access information relevant to their duties. This diagram highlights role-based access control and system functionalities.

2.2.2 Sequence Diagram

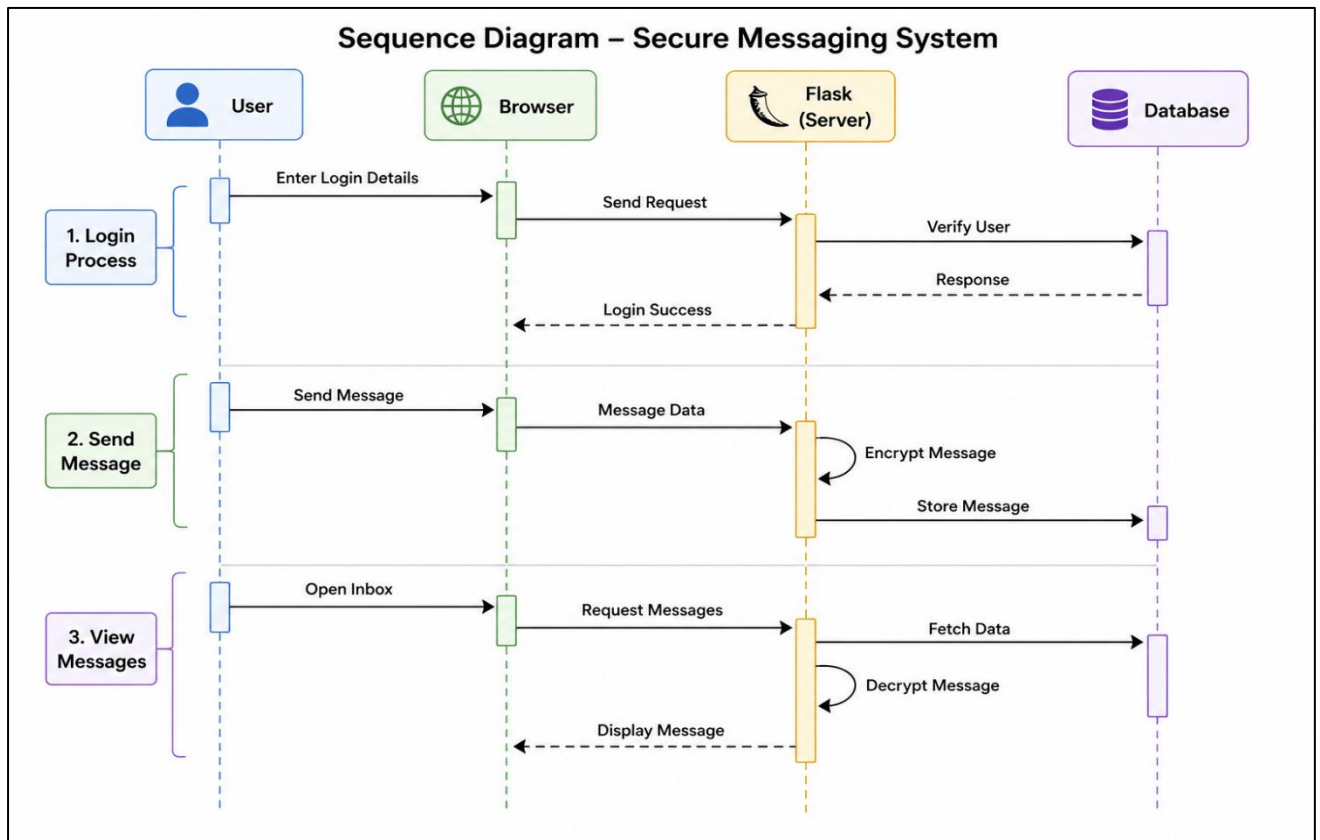


Fig: Sequence Diagram of Secure Messaging System

The sequence diagram illustrates the step-by-step flow of operations in the system. A user first logs in, and the request is verified by the server using the database. After successful login, the user can send a message, which is encrypted before being stored. When the user views a message, the system retrieves it from the database, decrypts it, and displays it. This diagram clearly shows the interaction between the user, browser, server, and database over time.

2.2.3 Class Diagram

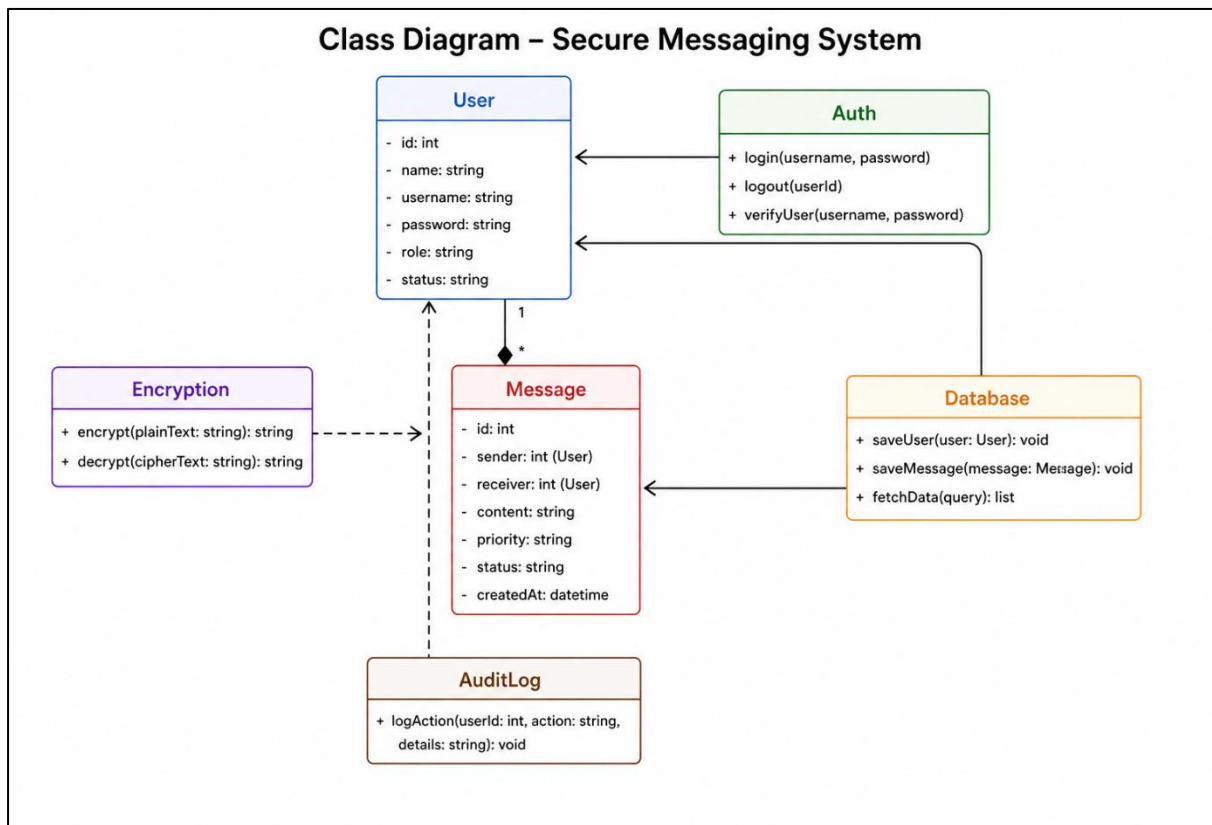


Fig: Class Diagram of Secure Messaging System

The class diagram represents the structure of the system by showing different classes and their relationships. The User class manages user details, while the Message class handles message data. The Auth class is responsible for authentication, and the Encryption class handles encryption and decryption processes. The Database class manages data storage, and the AuditLog class records system activities. This diagram helps in understanding the internal structure of the system.

2.2.4 Component Diagram

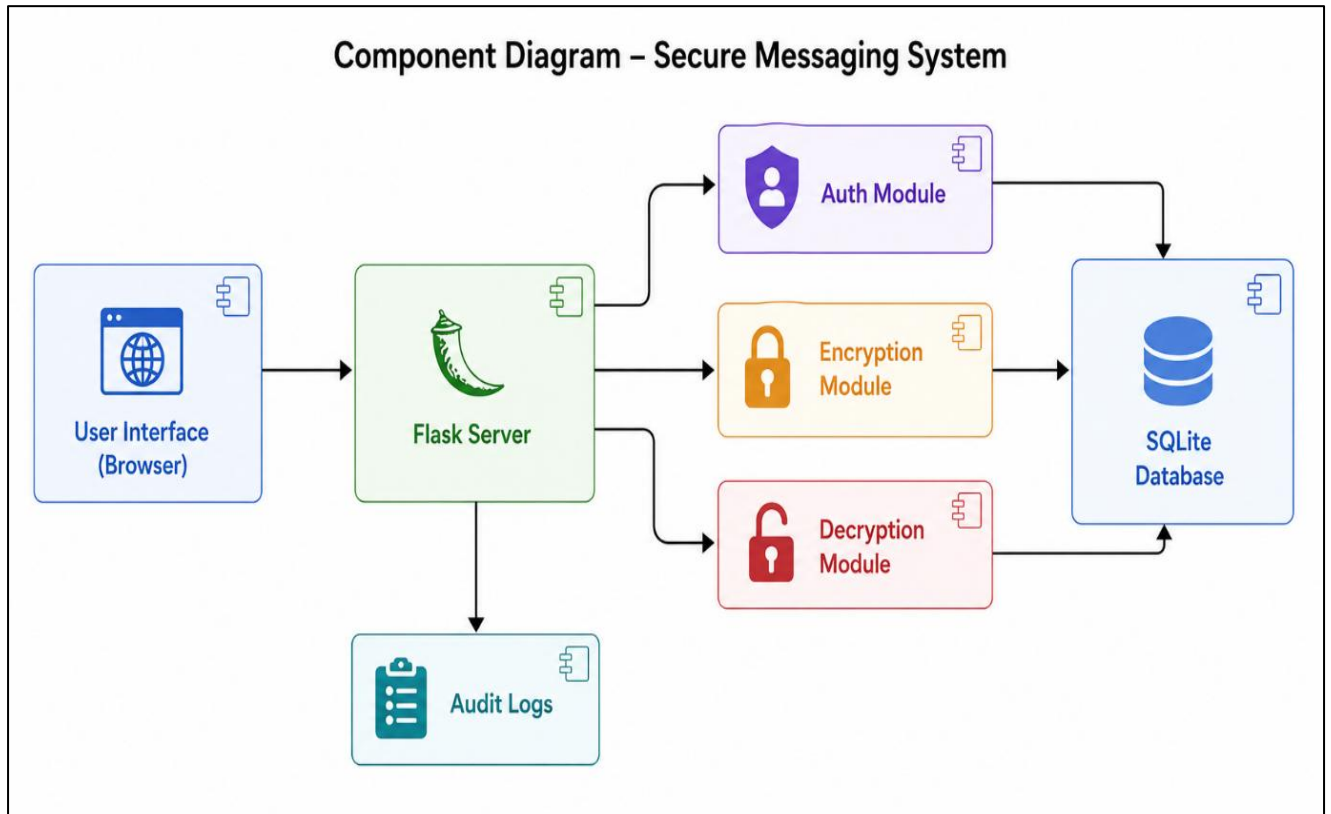


Figure: Component Diagram of Secure Messaging System

The component diagram illustrates the major modules of the system and their interactions.

The user interface communicates with the Flask server, which processes requests using modules such as authentication, encryption, and decryption. These modules interact with the SQLite database to store and retrieve data. The system also maintains audit logs to track user activities. This diagram provides a high-level view of the system architecture and module connectivity.

2.3 Implementation

Implementation Overview

The implementation phase converts the system design into a working application. The Secure Messaging System is developed using Python Flask for backend processing, SQLite3 for database storage, and HTML, CSS, and JavaScript for the frontend interface. The system ensures secure communication by encrypting messages using the Fernet encryption method before storing them.

Technology Stack Used

Component	Technology	Purpose
Backend	Python + Flask	Handles server logic and API requests
Database	SQLite3	Stores users, messages, and logs
Encryption	Fernet (Cryptography)	Secures messages
Frontend	HTML, CSS, JavaScript	User interface and interaction

Backend Implementation (Flask)

The Flask backend handles all system operations such as login, registration, message sending, and data retrieval. It verifies user credentials, manages sessions, encrypts messages before storing them, and decrypts them when required. API routes are created for each function like login, send message, and view messages.

Encryption Implementation

The system uses Fernet encryption to secure messages. Before storing any message, it is converted into an unreadable encrypted format. When an authorized user accesses the message, it is decrypted back to readable text using the same secret key. This ensures that sensitive data is never stored in plain text.

Database Implementation (SQLite3)

SQLite3 is used as a lightweight database to store all system data. It contains tables for users, messages, audit logs, and login tracking. The system performs operations like inserting new users, storing encrypted messages, retrieving messages, and updating message status.

Frontend Implementation

The frontend consists of web pages such as login, registration, dashboard, and admin panel. JavaScript is used to communicate with the backend through API calls. The interface allows users to send messages, view inbox, filter messages, and monitor system activity.

Session Management

Sessions are used to keep users logged in after authentication. The system stores user details like username and role in a session and automatically logs out the user after 30 minutes of inactivity for security purposes.

File Structure



2.4 Testing & Validation

Testing Overview

Testing was conducted to ensure that the system works correctly, securely, and efficiently in a real-world manufacturing environment. The main objective of testing was to verify that all functionalities operate as expected while maintaining strong data security.

The system was tested using different user roles such as Admin, Operator, QC Inspector, and Maintenance. This helped simulate actual usage scenarios and ensured that each role could perform its tasks without errors or unauthorized access.

Special attention was given to encryption, user authentication, and role-based access control, as these are the core features of the system.

Types of Testing Performed

The following testing methods were used to evaluate different aspects of the system:

- **Functional Testing** – to ensure all features work correctly
- **Security Testing** – to protect sensitive data and restrict access
- **Validation Testing** – to check correctness of user input
- **User Acceptance Testing (UAT)** – to evaluate ease of use
- **Integration Testing** – to verify interaction between components

Functional Testing

Functional testing was performed to validate all core features of the system.

- New users can register successfully and are marked as pending until approved
- Login works correctly for different account statuses (approved, pending, rejected)
- Users can send messages without any errors
- Messages are automatically encrypted before being stored in the database
- Only the intended receiver can access and decrypt the message
- Admin can approve users, manage records, and monitor system activity

Security Testing

Test Case	What was checked	Result
Encryption	Messages stored in encrypted format	Pass
Password Security	Password stored as hash	Pass
Authorization	Unauthorized access blocked	Pass
Session Timeout	Auto logout after inactivity	Pass

Validation Testing

Input	Invalid Case	Expected Result	Result
Username	Too short	Error message	Pass
Password	Mismatch	Error message	Pass
Message	Empty	Not allowed	Pass

User Acceptance Testing (UAT)

Role	Task	Result
Admin	Approve users, view logs	Pass
Operator	Send messages	Pass
QC	View reports	Pass
Maintenance	Handle requests	Pass

Integration Testing

Integration testing was carried out to ensure that all system components work together properly.

- The frontend successfully sends requests to the backend
- The backend processes requests correctly and applies business logic
- Encryption and decryption modules function without errors
- The database stores and retrieves data accurately
- Complete message flow (send → encrypt → store → retrieve → decrypt → display) works smoothly

2.5 Graphical User Interface (GUI) Layout

GUI Overview

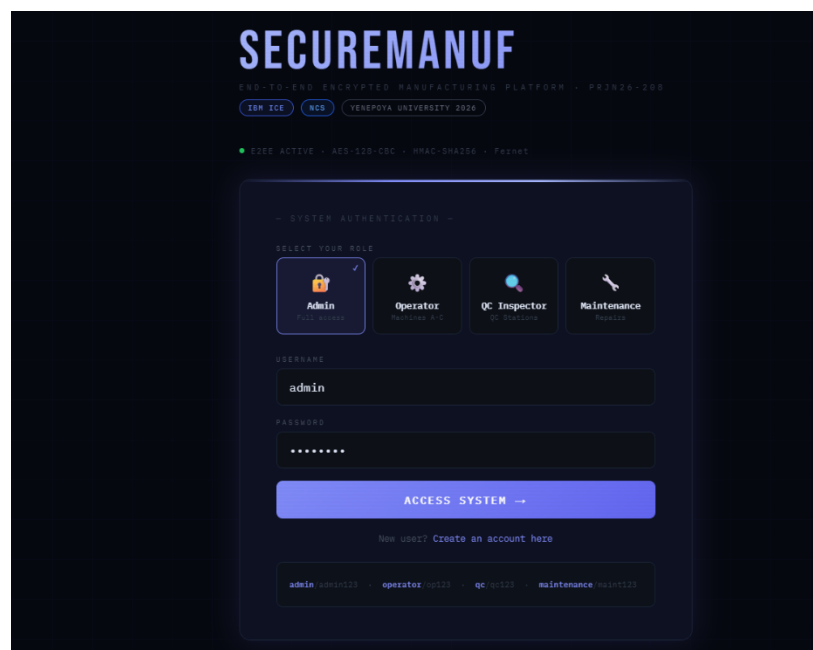
The Graphical User Interface is designed to be simple, user-friendly, and suitable for a manufacturing environment. A dark theme is used to reduce eye strain during long working hours. The interface is responsive and works across desktops, laptops, and tablets.

Pages in the System

Page	Purpose	Key Features
Login Page	User authentication	Role selection, username/password input, quick access
Register Page	New user registration	Form with validation, role selection
Dashboard	Main working interface	Statistics, send message, inbox, machine status
Admin Panel	System control (admin only)	User approval, audit logs, login tracking

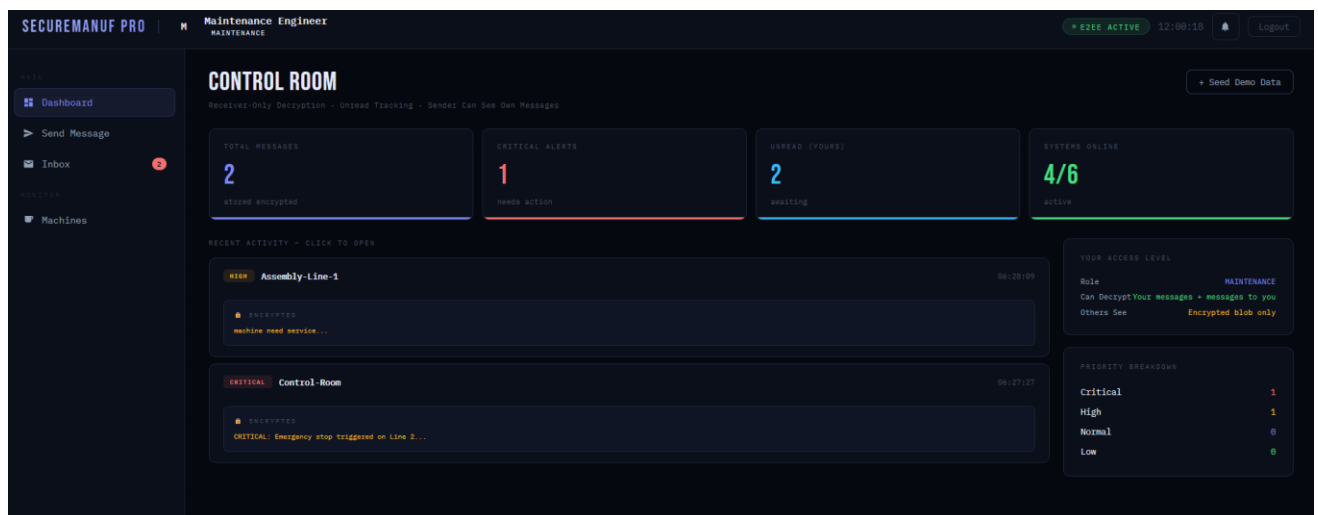
Login Page

- Role selection (Admin, Operator, QC, Maintenance)
- Username and password input fields
- Login button
- Link to registration page



Dashboard Page

- Top bar with user details and logout
- Sidebar for navigation
- Statistics section (messages, alerts)
- Send message panel
- Inbox section
- Machine status view



Admin Panel

- Approve or reject users
- View audit logs
- Monitor login activity

ADMIN PANEL

[Audit Logs](#)
[User Approvals](#)
[System Controls](#)

Refresh

PENDING USER APPROVALS

No pending approvals

AUDIT LOG

TIME	ACTION	DETAILS	USER
6:25:51 am	LOGIN_FAIL	Failed login attempt for qc	system
6:25:42 am	LOGIN_FAIL	Failed login attempt for operator	system
6:25:35 am	LOGOUT	qc logged out	qc
6:25:14 am	SEND	Added 10 demo messages	system
6:25:11 am	LOGIN	qc logged in from 127.0.0.1	qc
6:24:44 am	LOGIN	admin logged in from 127.0.0.1	admin

LOG IN TRACKING

qc logged in

IP: 127.0.0.1 | 20/6/2025, 6:25:11 am

Priority Colors and UI Theme

Definition:

Priority colors indicate the urgency of messages, while the UI theme defines the overall visual appearance of the system for better readability and user comfort.

Element	Color	Purpose
Critical Priority	Red	Indicates emergency or breakdown situations
High Priority	Yellow	Indicates urgent issues
Normal Priority	Blue	Indicates regular updates
Low Priority	Green	Indicates general information
Background	Dark Blue	Reduces eye strain
Cards/Sections	Dark Shade	Improves visual separation
Text	Light Gray	Enhances readability
Alerts	Red / Green	Shows error and success messages

2.6 Customer Testing

Customer Testing Overview

Customer testing is the process of evaluating the system using real users to ensure that it is practical, easy to use, and suitable for a real working environment. In this project, testing was conducted in a simulated manufacturing setup where users from different roles interacted with the system.

The main objective of customer testing is to check usability, user experience, and overall satisfaction. Unlike technical testing, which focuses on system performance and security, customer testing focuses on how effectively users can perform their daily tasks using the system.

Four participants representing key roles—Admin, Operator, QC Inspector, and Maintenance staff—were selected. Each participant performed common tasks such as logging in, sending messages, reading encrypted messages, and using role-specific features. Observations were made on task completion, ease of use, and time taken.

Tasks Performed and Results

Task	Description	Result
Login	Access system using credentials	Successful
Registration	Create new user account	Successful
Send Message	Send encrypted message with priority	Successful
Read Message	Decrypt and view message	Successful
Filter Messages	Sort messages by priority	Successful
Logout	Exit the system securely	Successful
Admin Actions	Approve users and view logs	Successful

2.7 Evaluation

Evaluation Overview

Evaluation is carried out to determine how effectively the system meets the project objectives and performs in a practical environment. The Secure Messaging System was evaluated based on functionality, security, performance, usability, and overall effectiveness in the manufacturing unit.

The evaluation confirmed that the system successfully encrypts messages, controls user access, maintains logs, and provides a simple interface for communication. All major modules function as expected, and the system performs efficiently under normal operating conditions.

Although the system meets all major requirements, certain enhancements such as real-time notifications and message search can improve usability in future versions.

Table: System Evaluation Summary

Criteria	Evaluation
Functionality	Successfully performs secure registration, login, encryption, and messaging
Security	Protects messages using encryption and restricts unauthorized access
Performance	Responds quickly and operates efficiently
Usability	Provides a simple and easy-to-use interface
Reliability	All major functions work consistently without errors
Scalability	Can be improved further with advanced storage and live notifications

3. PROJECT SNAPSHOTS (UI & OUTPUT SCREENS)

3.1 Login Page

The login page is the entry point of the system where users enter their credentials to access the application. It also provides role selection and a link to the registration page.

Key Features:

- Username and password input
- Role selection (Admin, Operator, QC, Maintenance)
- Login button
- Register link

SECUREMANUF
END-TO-END ENCRYPTED MANUFACTURING PLATFORM · PRJN26-208

IBM ICE NCS YENEPOYA UNIVERSITY 2026

● E2EE ACTIVE · AES-128-CBC · HMAC-SHA256 · Fernet

— SYSTEM AUTHENTICATION —

SELECT YOUR ROLE

Admin
Full access

Operator
Machines A-C

QC Inspector
QC Stations

Maintenance
Repairs

USERNAME
admin

PASSWORD
.....

ACCESS SYSTEM →

New user? [Create an account here](#)

admin/admin123 · operator/op123 · qc/qc123 · maintenance/maint123

Fig 1: Login Page

3.2 Registration Page

The registration page allows new users to create an account. Newly registered users must wait for admin approval before accessing the system.

Key Features:

- Full name, username, password fields
- Role selection
- Register button

SECUREMANUF
END-TO-END ENCRYPTED MANUFACTURING PLATFORM
● E2EE ACTIVE · [Register New Account](#)

— CREATE NEW ACCOUNT —

FULL NAME
Enter your full name

USERNAME
Choose a username

PASSWORD
Create a password (min 4 chars)

CONFIRM PASSWORD
Confirm your password

SELECT ROLE
👤 Floor Operator (Machines A-C, Assembly Line) ▼

REGISTER →

Already have an account? [Login here](#)

Fig 2: Registration Page

3.3 Dashboard

The dashboard is the main interface after login. It provides access to all system features and displays important information.

Key Features:

- Sidebar navigation
- Message statistics
- Quick access to send, inbox, and machines

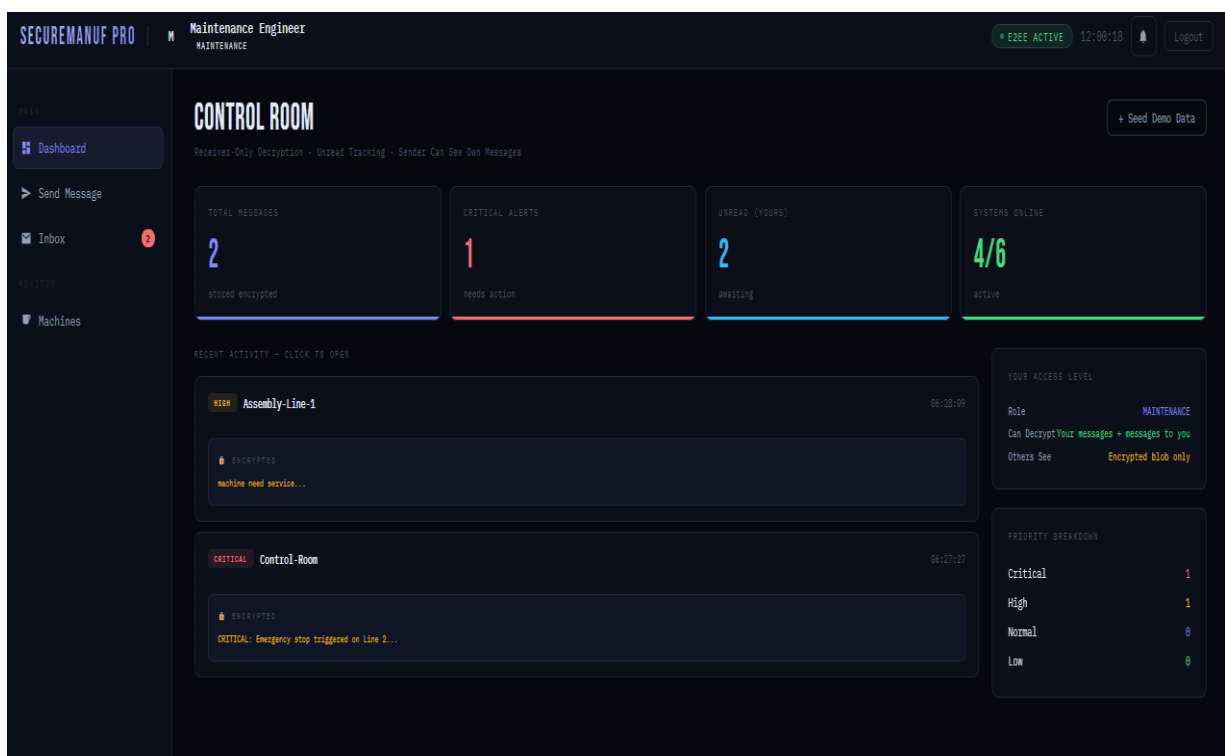


Fig 3: Dashboard Interface

3.4 Send Message

This section allows users to compose and send encrypted messages with priority levels.

Key Features:

- Message input
- Receiver selection
- Priority selection
- Encrypt and send option

SECUREMANUF PRO | QC Inspector
qc

E2EE ACTIVE 16:18:36 Logout

SEND MESSAGE
Encrypted before storage - plaintext never saved

FROM (SENDER) TO (RECEIVER) QUICK TEMPLATE

QC-Station-2 Machine-A Emergency stop

MESSAGE CONTENT

CRITICAL: Emergency stop triggered on Line 2

Normal Low High Critical

ENCRYPT & SEND → Only receiver and sender can decrypt.

Fig 4: Send Message Panel

3.5 Inbox

The inbox displays all messages related to the user. Messages can be filtered and opened for decryption.

Key Features:

- Message list
- Priority indicators
- Filter options

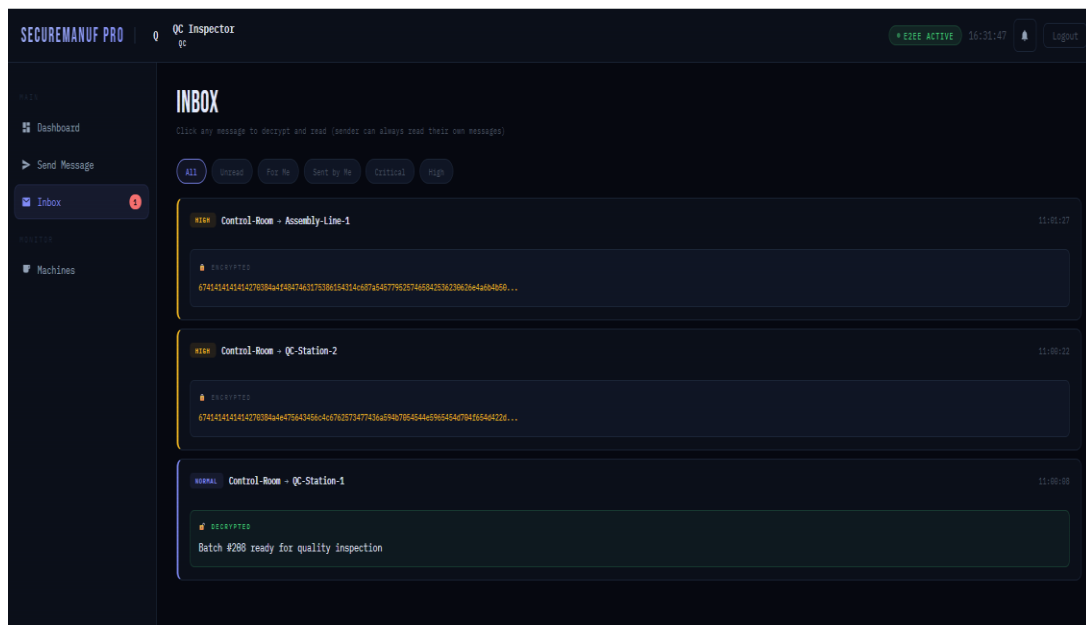


Fig 5: Inbox Panel

3.6 Message Decryption

When a message is opened, it is decrypted and displayed to the authorized user.

Key Features:

- Sender and receiver details
- Decrypted message content
- Security validation

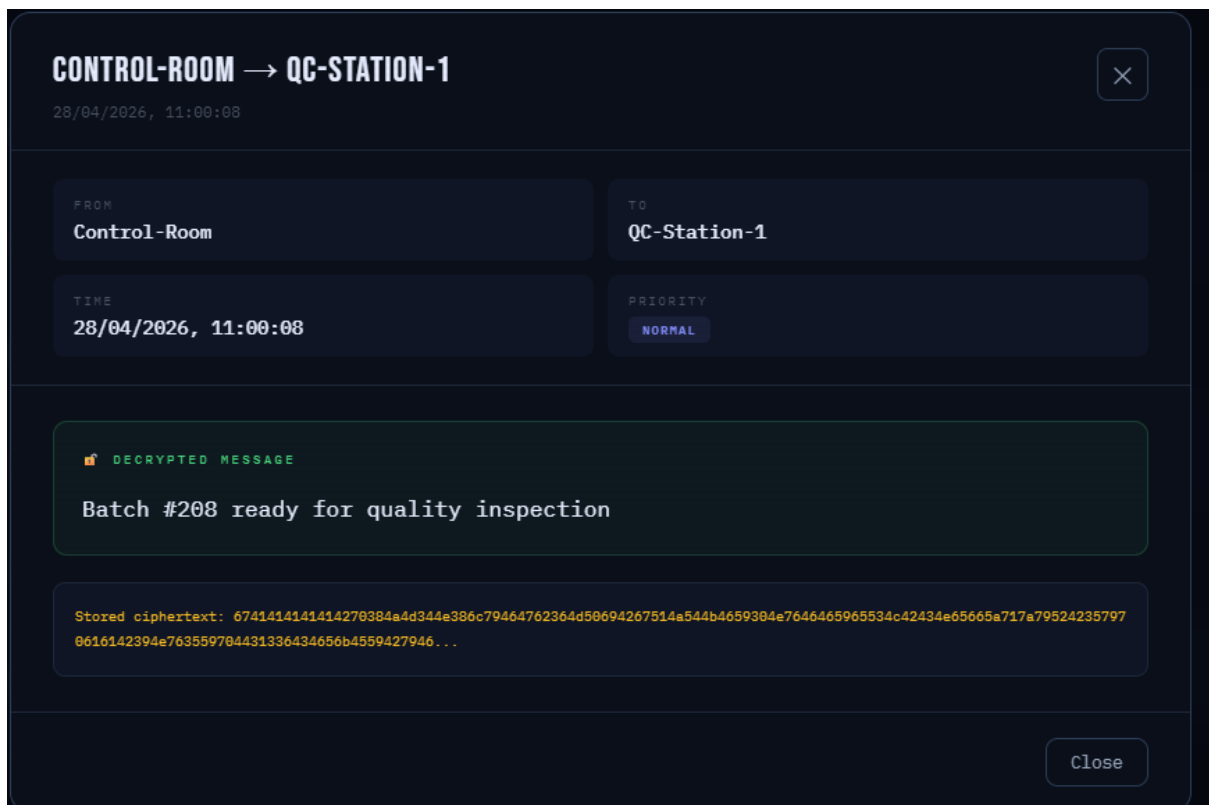


Fig 6: Message Decryption View

3.7 Admin Panel

The admin panel allows system management and monitoring.

Key Features:

- User approval
- Audit logs
- Login tracking

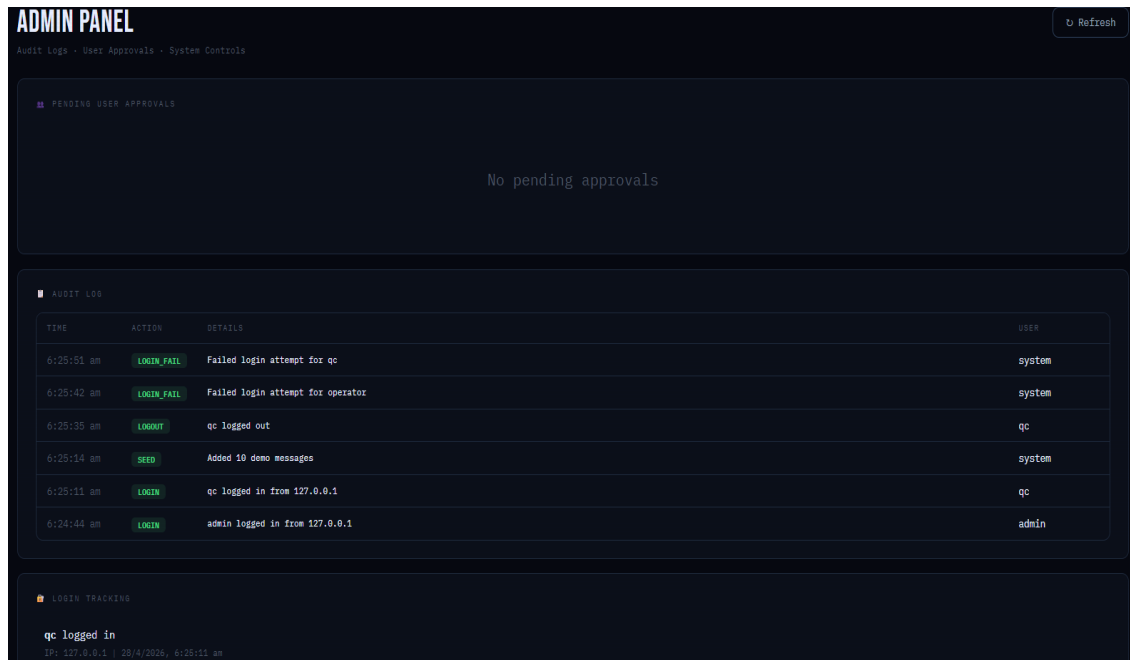


Fig 8: Admin Panel

4. CONCLUSION

The *End-to-End Encryption for Manufacturing Unit* project was successfully designed and implemented to provide a secure communication system within a factory environment. The main objective of the project was to protect sensitive information such as machine alerts, quality reports, and maintenance updates from unauthorized access.

The system ensures that all messages are encrypted using the Fernet algorithm before being stored in the database. This guarantees that no plaintext data is stored, and only the sender and the intended receiver can decrypt and read the messages. This strong security mechanism makes the system reliable for handling confidential production data.

All the planned objectives of the project were successfully achieved. The system supports role-based access for Admin, Operator, QC Inspector, and Maintenance users, ensuring that each user has appropriate permissions. The admin approval feature adds an extra layer of control by allowing only verified users to access the system. Additionally, audit logging and login tracking provide transparency and help monitor all user activities.

The system performs efficiently, with fast response times for login, message sending, and message decryption. The user interface is simple, clean, and easy to use, even for workers with basic computer knowledge. The dark theme and organized layout further improve usability in a manufacturing environment.

However, the system has some limitations, such as the absence of real-time notifications, message search functionality, and file sharing. These features can be added in future versions to enhance usability and scalability. Implementing advanced security features like per-user encryption keys can further strengthen the system.

In conclusion, the project successfully demonstrates a secure, reliable, and user-friendly messaging system tailored for manufacturing units. It meets all functional and security requirements and is suitable for real-world deployment in small to medium-scale industries. The system provides a strong foundation for future improvements and advanced features.

5. FUTURE DEVELOPMENT

Future Development Overview

The current system successfully provides secure communication using end-to-end encryption for a manufacturing unit. However, there is scope for further improvement to enhance usability, security, and scalability. Based on user feedback and practical requirements, several future enhancements have been identified.

Planned Enhancements

The following features can be added in future versions to improve the system:

- **Real-time notifications:** Enable instant message updates without refreshing the page.
- **File attachments:** Allow users to send images, reports, and documents securely.
- **Search functionality:** Help users quickly find old messages.
- **Message forwarding:** Allow sharing of important information across users.
- **Delete messages:** Enable users to remove unwanted or incorrect messages.
- **Mobile application:** Provide access through smartphones for better accessibility.
- **Language support:** Support regional languages for ease of use.

Security Improvements

To further strengthen the system, the following security enhancements can be implemented:

- **Per-user encryption keys** to improve data protection
- **Two-factor authentication (2FA)** for secure login
- **Account lock after failed attempts** to prevent unauthorized access
- **Database encryption** for additional security layer

User Experience Improvements

The system can be made more user-friendly with:

- Push notifications for alerts
- Message templates for quick communication
- Draft saving to prevent data loss
- Profile customization options

9. References

<i>Reference</i>	<i>Source URL</i>	<i>Used For</i>
<i>Python Official Documentation</i>	https://docs.python.org/3/	Python programming language syntax and functions
<i>Flask Framework Documentation</i>	https://flask.palletsprojects.com/	Web framework for backend development
<i>Cryptography (Fernet) Documentation</i>	https://cryptography.io/en/latest/fernet/	Encryption and decryption implementation
<i>SQLite3 Documentation</i>	https://sqlite.org/docs.html	Database design and SQL queries
<i>HTML5 and CSS3 Documentation</i>	https://developer.mozilla.org/	Frontend user interface development
<i>JavaScript Documentation</i>	https://developer.mozilla.org/en-US/docs/Web/JavaScript	Client-side form validation and API calls
<i>Mermaid.js Documentation</i>	https://mermaid.js.org/	Creating flowcharts and diagrams
<i>OWASP Authentication Cheat Sheet</i>	https://cheatsheetseries.owasp.org/	Session management and security guidelines

7. APPENDIX

7.1 Sample Encrypted Messages

The table below demonstrates how plaintext messages are converted into ciphertext using the Fernet encryption algorithm. The system stores only encrypted data in the database, ensuring message confidentiality.

Plaintext Message	Encrypted Message (Ciphertext)
Machine A temperature is too high	gAAAAABmMx7L5tXk3QvR9p2YwB8nKqXcVzL9p2Yw==
Batch #208 passed quality inspection	gAAAAABnYz8M6uYl4RwS0q3ZxC9mPwD1aFbG2hIjK3lMnOpQrStUvWxYz==
Maintenance required on Assembly Line	gAAAAABoZ9N7OvPqR1sT2uV3wX4yZ5aB6cC7dD8eE9fF0gG1hH2iI3jJ4kK==
Critical alert: Machine B stopped	gAAAAABpA0O8PwQrS1tU2vW3xY4zA5bC6dD7eE8fF9gG0hH1iI2jJ3kK4lL==
Quality check failed for Batch #209	gAAAAABqB1P9QxRrS2uV3wX4yZ5aB6cC7dD8eE9fF0gG1hH2iI3jJ4kK5lL==

Note: Fernet encryption generates different ciphertext each time for the same plaintext due to the use of a random initialization vector (IV).

7.2 Database Schema

The system uses SQLite to store users, messages, and logs.

Users Table

- Stores all user account details
- Includes role and approval status

Messages Table

- Stores encrypted messages
- Includes sender, receiver, priority, and read status

Audit Log Table

- Records all system activities such as login, message sending, and approvals

Login Tracking Table

- Tracks login attempts with timestamps and after-hours flag

7.3 System Output Screens

The following screens are part of the system interface:

Screen	Description
Login Screen	User authentication interface
Registration Screen	New account creation
Dashboard	Main interface with navigation and statistics
Send Message	Compose and send encrypted messages
Inbox	View and filter messages
Decryption View	Displays decrypted message content
Admin Panel	User management and audit logs

7.4 Glossary of Terms

- **End-to-End Encryption:** A security method where messages are encrypted at the sender side and can only be decrypted by the intended receiver, ensuring complete confidentiality.
- **Plaintext:** The original readable form of data or message before encryption is applied.
- **Ciphertext:** The encrypted and unreadable form of a message after applying encryption.
- **Encryption Key:** A secret value used to convert plaintext into ciphertext and vice versa.
- **Fernet:** A symmetric encryption method that ensures secure message encryption and authentication.
- **Hashing:** A one-way process of converting data (such as passwords) into a fixed-length value that cannot be reversed.
- **Session:** A temporary storage mechanism used to maintain user login state during interaction with the system.
- **Role-Based Access Control:** A security approach where users are given permissions based on their roles.
- **Audit Log:** A record of system activities used for tracking user actions and maintaining security.
- **API:** A set of endpoints that allow communication between the frontend and backend of the system.
- **JSON:** A lightweight data format used for exchanging data between client and server.

7.5 Abbreviations

List of Abbreviations

Abbreviation	Full Form	Description
API	Application Programming Interface	Enables communication between frontend and backend components
AES	Advanced Encryption Standard	Encryption algorithm used internally by Fernet
SHA-256	Secure Hash Algorithm (256-bit)	Used for securely hashing user passwords
HTML	HyperText Markup Language	Used to structure web pages
CSS	Cascading Style Sheets	Used to style and design the user interface
JS	JavaScript	Used for frontend logic and API communication
SQL	Structured Query Language	Used to manage and query the database
HTTP	HyperText Transfer Protocol	Protocol for communication between client and server
HTTPS	HyperText Transfer Protocol Secure	Secure version of HTTP used in production environments
JSON	JavaScript Object Notation	Format for sending and receiving data between client and server
DB	Database	Refers to SQLite database used for data storage
UI	User Interface	The visual interface through which users interact with the system



Innovation Centre for Education

